

РЕШЕН ИЗПИТ С ПЪЛНИ ОБЯСНЕНИЯ

- Вариант 2

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ НА ЗАДАЧАТА

Методичен разбор на изискванията, за да подготвим архитектурата на решението.

ИЗРАЗ ОТ УСЛОВИЕТО	КАКВО ОЗНАЧАВА ЗА НАС В С
"вид на курса (могат да бъдат два вида: Програмен или Дизайн)"	int type; // 0 = Програмен, 1 = Дизайн Можем да използваме enum или просто цяло число (флаг), за да спестим памет и да улесним проверките (спрямо strcmp с низове).
"данните се съхраняват в бинарни файлове... прочетете ги в масиви"	fread цикъл с разширяване на масива (чрез realloc). Трябва да броим колко елемента сме прочели.
"запис на нов ред: <id>;<fac>;<date>;<price>"	fprintf(f, "%lld;%lld;%s;%.2f\n", ...); Текстов файл, добавяне (append) с режим "a". Използваме точка и запетая като разделител.
"върща като резултат всички курсове... по-малка от средната"	Функцията трябва да върне нов динамичен масив (указател Course*). Трябват два цикъла: един за средната цена и втори за заделяне на памет и копиране.

СЕКЦИЯ 1 — ПЪЛЕН КОД

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// --- СТРУКТУРИ ---
typedef struct {
    long long id;
    char name[129];
    char date[11]; // mm/dd/yyyy
    float price;
```

```
int students_count;
int type; // 0 за Програмен, 1 за Дизайн
} Course;
```

```
typedef struct {
    char name[81];
    long long fac_num;
    int courses_count;
    long long *course_ids;
} Student;
```

```
// --- ПОМОЩНА ФУНКЦИЯ ЗА ДАТИ ---
```

```
int compareDates(const char* date1, const char* date2) {
    int m1, d1, y1, m2, d2, y2;
    sscanf(date1, "%d/%d/%d", &m1, &d1, &y1);
    sscanf(date2, "%d.%d.%d", &d2, &m2, &y2); // Внимание: Параметърът в Зад. 2 е
    dd.mm.yyyy!
```

```
    if (y1 != y2) return y1 - y2;
    if (m1 != m2) return m1 - m2;
    return d1 - d2;
}
```

```
// --- ЗАДАЧА 1: Четене от бинарен файл в динамичен масив ---
```

```
Course* readCoursesBinary(int *count) {
    FILE *f = fopen("courses.bin", "rb");
    if (!f) return NULL;

    Course *arr = NULL;
    Course temp;
    *count = 0;

    // Четем елемент по елемент до достигане края на файла
    while (fread(&temp, sizeof(Course), 1, f) == 1) {
        arr = realloc(arr, (*count + 1) * sizeof(Course));
        if (!arr) { fclose(f); return NULL; }
        arr[*count] = temp;
        (*count)++;
    }
}
```

```
fclose(f);
return arr;
}
```

// --- ЗАДАЧА 2: Изтриване по критерии ---

```
void deleteCourses(Course **courses, int *count, int target_type, const char *target_date) {
    int new_count = 0;
    int deleted = 0;
```

```
    for (int i = 0; i < *count; i++) {
        // Условие за изтриване:
        // 1. Същият вид
        // 2. Студенти < 8
        // 3. Начална дата СЛЕД подадената
        int should_delete = 0;
        if ((*courses)[i].type == target_type && (*courses)[i].students_count < 8) {
            if (compareDates((*courses)[i].date, target_date) > 0) {
                should_delete = 1;
            }
        }
    }
}
```

```
    if (!should_delete) {
        (*courses)[new_count] = (*courses)[i];
        new_count++;
    } else {
        deleted++;
    }
}
```

```
    if (deleted == 0) {
        printf("Няма открито съвпадение за изтриване.\n");
    } else {
        if (new_count > 0) {
            *courses = realloc(*courses, new_count * sizeof(Course));
        } else {
            free(*courses);
            *courses = NULL;
        }
    }
}
```

```
    *count = new_count;
}
}
```

// --- ЗАДАЧА 3: Запис на студент (Текстов файл) ---

```
int enrollStudentTxt(Course *courses, int c_count, Student *students, int s_count, long long
fac_num, long long course_id) {
    Student *s = NULL; Course *c = NULL;
```

```
    for(int i=0; i<s_count; i++) { if(students[i].fac_num == fac_num) s = &students[i]; }
    for(int i=0; i<c_count; i++) { if(courses[i].id == course_id) c = &courses[i]; }
```

```
    if(!s || !c) return -1;
```

```
    if (c->students_count >= 50) return 0;
    if (s->courses_count >= 7) return 0;
```

```
    for(int i=0; i<s->courses_count; i++) {
        if(s->course_ids[i] == course_id) return 0;
    }
```

// Пишем в текстов файл

```
FILE *f = fopen("students_info.txt", "a");
if (!f) return -1;
fprintf(f, "%lld;%lld;%s;%.2f\n", c->id, s->fac_num, c->date, c->price);
fclose(f);
```

```
c->students_count++;
s->course_ids = realloc(s->course_ids, (s->courses_count + 1) * sizeof(long long));
s->course_ids[s->courses_count] = course_id;
s->courses_count++;
```

```
return 1;
```

```
}
```

// --- ЗАДАЧА 4: Връщане на филтриран масив ---

```
Course* getAffordableCourses(Course *courses, int count, int *result_count) {
    if (count == 0) return NULL;
```

```

float sum = 0;
for (int i = 0; i < count; i++) {
    sum += courses[i].price;
}
float avg = sum / count;

Course *result = NULL;
*result_count = 0;

for (int i = 0; i < count; i++) {
    if (courses[i].students_count < 50 && courses[i].price < avg) {
        result = realloc(result, (*result_count + 1) * sizeof(Course));
        if (!result) return NULL; // Грешка при заделяне
        result[*result_count] = courses[i];
        (*result_count)++;
    }
}

return result; // Ако няма курсове, ще върне NULL, което е търсеното поведение
}

```

СЕКЦИЯ 2 — ОБЯСНЕНИЯ СЪПКА ПО СЪПКА

Разчитане на Бинарни Файлове (Задача 1)

Защо използваме `while (fread(...) == 1)`? Функцията `fread` чете блокове от памет (в нашия случай блок с размер `sizeof(Course)`). Ако успешно прочете един пълен блок, връща 1. Когато стигне края на файла (EOF), ще върне 0. Това ни дава перфектното условие за цикъл: четем един елемент в буфер `temp`, ако е успешно — разширяваме масива с `realloc` и го записваме там.

Аналогия от реалния свят: Представете си, че вадите тухли от камион една по една, без да знаете колко са общо. Всеки път, когато хванете тухла (`fread == 1`), добавяте място за нея на строежа (`realloc`) и я слагате.

Филтриране и Нов Масив (Задача 4)

Когато условието гласи "Функцията връща като резултат всички курсове", това почти винаги означава заделяне на **нов масив**. Използваме класическия алгоритъм с два цикъла:

1. **Цикъл 1:** Изчисляваме средната цена. Трябва да обходим всички елементи, за да натрупаме сумата, и накрая делим на броя. (Внимаваме за деление на нула!).

2. **Цикъл 2:** Обхождаме отново. Ако елементът отговаря на условията (`price < avg` и `students_count < 50`), ние `realloc`-ваме новия масив с едно място повече и копираме елемента.

СЕКЦИЯ 3 — ТОП 3 ГРЕШКИ В ТОЗИ ВАРИАНТ

- **Грешка 1: Форматите на датите се разминават.** В Задача 1 датата е във формат `mm/dd/yyyy` (със слаш), но в Задача 2 параметърът е `dd.mm.yyyy` (с точки). Ако използвате сляпо една и съща функция за парсване, програмата ще крашне или ще работи грешно! Нужно е да съобразим `sscanf` логиката с конкретните разделители (вижте функцията `compareDates`).
- **Грешка 2: NULL проверка при `realloc` в цикъл.** В Задача 1 четем бинарния файл в цикъл. Ако `realloc` върне `NULL` (няма памет), оригиналният указател не се губи, но трябва незабавно да затворим файла `fclose(f)` преди да върнем грешка, иначе заключваме файла на операционната система (Resource Leak).
- **Грешка 3: "Не връща резултат" vs "Връща масив".** Много студенти бъркат сигнатурите. В Задача 2 изтриването е `void` (или променя масива чрез двоен указател `**`), докато в Задача 4 се очаква `Course*` (указател към първия елемент на новосъздаден масив). Разликата е критична за компилацията.